

Rust for finantial application

A "high-level" language for complex financial applications/services

The two qualities we're after

Maintainability

Software that can be **changed easily and safely for years**.

Correctness

Software that does what the domain says — and **cannot be made to do otherwise**.

Often you can't have both. The rest of this talk argues that the DDD/Onion/Hexagonal architecture **plus Rust** gives you both.

Type Driven Development (TyDD)

Modeling the domain in types — make illegal states unrepresentable

The building blocks

Just enough Rust to read the next slides

Primitive types

Rust	≈ Ruby	Notes
`u64`	`Integer`	unsigned — **cannot go negative**
`f64`	`Float`	64-bit floating point
`String`	`String`	owned, growable text
`Vec<T>`	`Array`	growable list; `T` = element type

`struct` — a product type ("**AND**")

```
struct Customer {
    name: String,         // every Customer has a name,
    age:  u64,            // AND an age,
    tags: Vec<String>,    // AND a list of tags
}
```

- All fields, all the time — like a Ruby `Struct` / `Data`, with types checked at compile time

Enums + exhaustive `match`

An `enum` is a choice type (a.k.a. **sum type**) — a value is one variant **OR** another:

```
enum Fee {
    FlatPennies(u64),
    Percentage(f64),
}

fn apply(fee: &Fee, amount_pennies: u64) -> u64 {
    match fee {
        Fee::FlatPennies(p) => amount_pennies + p,
        Fee::Percentage(r) => amount_pennies
            + (amount_pennies as f64 * r).round() as u64,
    }
}
```

Add a new variant tomorrow → **every** `match` that must change fails to compile — the compiler forces you to handle it.

Absence and failure, made explicit

`Option` istead of `nil`

```
pub struct User {
    id:    u64,
    email: Option<String>,
}

match user.email {
    Some(email) => /* send receipt */,
    None        => /* no email on file */,
}
```

No accidental `NoMethodError` on `nil`.

`Result` instead of exceptions

```
fn settle(p: PaidPayment)
    -> Result<Settlement, SettleError>;

let settlement = settle(payment)?;

// `?` is sugar for:
let settlement = match settle(payment) {
    Ok(s)  => s,
    Err(e) => return Err(e.into()),
};
```

Failure is **part of the signature**, and callers cannot silently ignore it.

Newtype pattern — parse, don’t validate

```rust
pub struct PaymentId(u64); // field is private to the module

impl PaymentId {
    pub fn new(raw: u64) -> Result<Self, PaymentIdError> {
        if raw == 0 {
            Err(PaymentIdError::Zero)
        } else {
            Ok(Self(raw))
        }
    }
    pub fn as_u64(&self) -> u64 { self.0 }
}

// example of use — the ids are distinct types, so they cannot be swapped
fn accredite(payment: PaymentId, to: AccountId) { /* ... */ }

// instead of
fn accredite(payment: u64, to: u64) { /* ... */ }
```

Validation happens **once**, at the boundary. Afterwards the type *is* the proof.

Typestate pattern — payment lifecycle

```
pub struct DraftPayment  { id: PaymentId, amount: Money }
pub struct PlacedPayment { id: PaymentId, amount: Money }
pub struct PaidPayment   { id: PaymentId, amount: Money }

impl DraftPayment {
    pub fn place(self) -> PlacedPayment { /* ... */ }
}

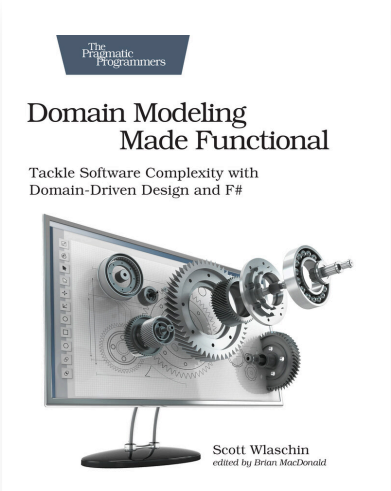
impl PlacedPayment {
    pub fn pay(self) -> PaidPayment { /* ... */ }
    pub fn void(self) -> DraftPayment { /* ... */ }
}

placed_payment.pay(); // this compiles
draft_payment.pay();  // does not compile
```

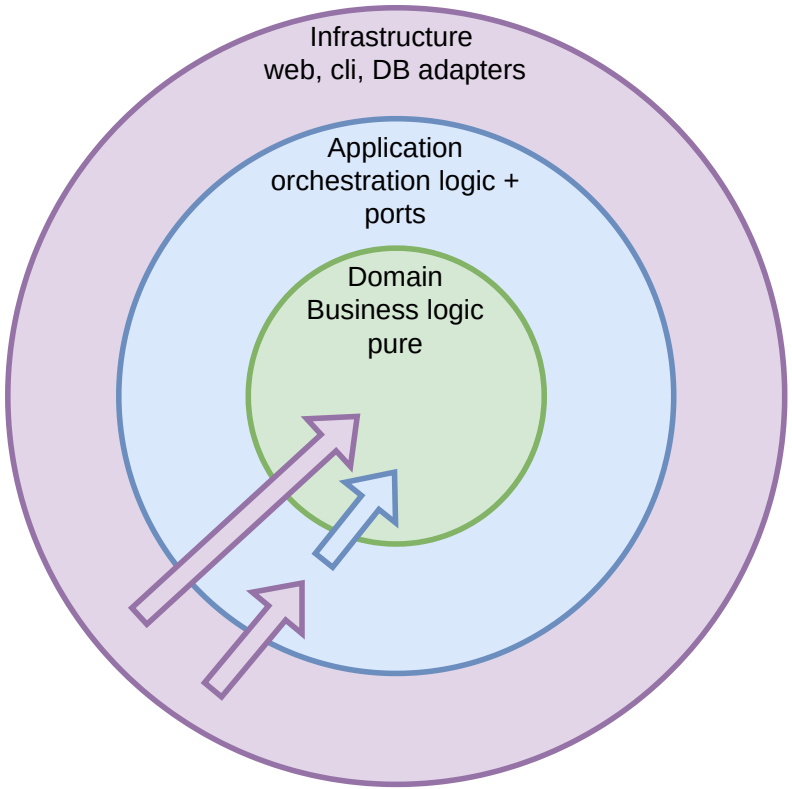
The entire state machine is checked at compile time.

Architecture: Domain Driven Design

Inspired by *"Domain Modeling Made Functional"* (Scott Wlaschin), via Onion/Hexagonal architecture



Onion/Hexagonal architecture



- **Domain** — the business logic, expressed as types, pure functions. No I/O, no framework.
- **Application** — Orchestration logic. Declares the **ports** the infrastructure implements.
- **Infrastructure** — **Driving adapters** (e.g. CLI, HTTP) that call the application, and **driven adapters** that implement the ports.

Dependencies point **inward only** — a *functional core* with an *imperative shell*.

The dependency rule

- **Ports** are traits defined by the inner layers; **adapters** implement them on the outside.

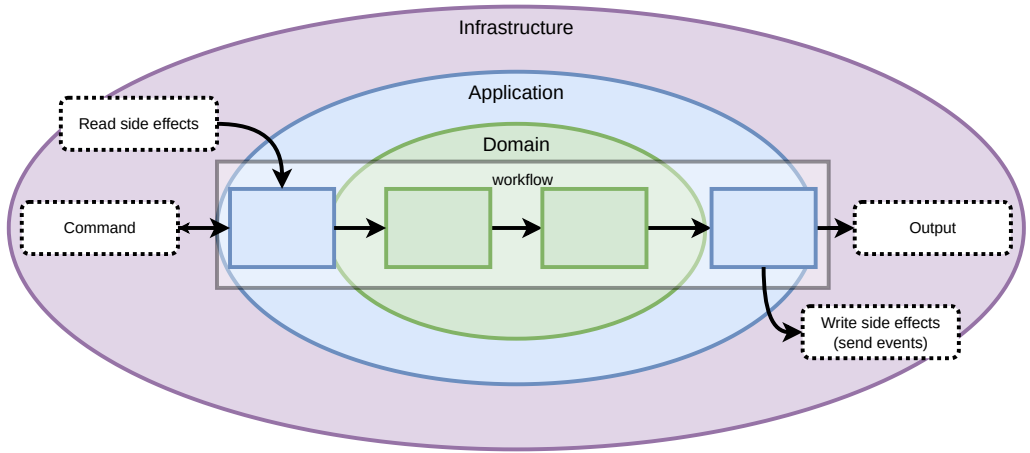
```
// In Application layer — the port
trait PaymentRepository {
    async fn find_payment(&self, id: &PaymentId) -> Result<Payment, RepoError>;
}

// In Infrastructure layer — the adapter
struct MySqlPaymentRepository { pool: MySqlPool }

impl PaymentRepository for MySqlPaymentRepository {
    async fn find_payment(&self, id: &PaymentId) -> Result<Payment, RepoError> { /* ... */ }
}
```

- The domain and application layers know nothing about HTTP, SQL, event bus, external services.
- Swapping MySQL for an in-memory store, or HTTP for a CLI, touches **only infrastructure**.
- **Enforced by Rust at compile time** — We split the app into workspace crates (`domain` , `application` and `infrastructure`) and define the dependency among them. Rust enforce the dependencies at compile time.

Workflow



- A workflow is one business process (`place_payment` , `settle` , `refund`). Use case in Clean Architecture.
- We split it in two: a **pure Domain workflow** (the decision) and an **Application workflow** (the orchestration + I/O via external adapters).
- The **domain** decides *what* should happen; the **application** makes it happen — loading state, persisting it, publishing events.

Domain workflow — pure

`(State, Command) -> (State, Events)`

Application workflow — I/O

`Command -> Output` , I/O via adapters

Domain workflow — pure

- A **pure function**: `(State, Command) -> (State, Events)`.
- Encodes the business **decision** — no I/O, no framework, no async.
- Deterministic: the same state + command always gives the same state + events.
- Trivially testable — no fakes, no mocks, no database.

```
// (State, Command) -> (State, Events)
pub fn place_payment(state: DraftPayment, cmd: PlacePayment)
    -> Result<(PlacedPayment, Vec<DomainEvent>), DomainError>
{
    let payment = state.place(cmd)?;           // DraftPayment -> PlacedPayment (command drives the decision)
    let events  = vec![DomainEvent::PaymentPlaced(payment.id.clone())];

    Ok((payment, events))
}
```

Application workflow — orchestration & I/O

- **Orchestrates** the domain workflow and owns all **I/O**, behind **ports**.
- Loads state → runs the pure domain workflow → persists the new state and publishes the events.
- Persisting state and sending events can happen in the **same transaction** (unit of work / outbox) — no lost events.
- Dependencies are **injected as ports**, so the workflow is testable with fakes.

```rust
pub async fn place_payment(
    cmd: PlacePayment,
    repo: &impl PaymentRepository,
    uow: &impl PaymentUnitOfWork,
) -> Result<PlacedPayment, PlaceError> {
    let valid_cmd       = validate(cmd)?;                          // boundary validation
    let state           = repo.find_payment(&valid_cmd.id).await?; // I/O via port
    let (state, events) = domain::place_payment(state, valid_cmd)?; // pure domain workflow
    uow.commit(&state, &events).await?;                            // state + events, one tx
    Ok(state)
}
```

Safe concurrency

- Our services are mostly **I/O-bound** — many concurrent requests, DB calls, external services
- Rust handles them with **async/await** on **Tokio**: lightweight tasks, not a thread per request
- Rust gives **compile-time data-race freedom** — *"fearless concurrency"* without a GIL (Global Interpreter Lock) or GC, while still allowing in-place mutation

Caveat: this rules out **data races**, not **logic races or deadlocks** — those are still your job.

Correctness is important

In fintech, a bug is not a glitch — it is money

Traditional correctness tools

- **Discipline** — conventions, reviews, care
- **Unit / integration / acceptance tests**
- **Syntax linting**
- **QA**

Correctness toolbox

Seen so far

- **Compiler checks** — call a function that doesn't exist? error at compile time
- **Type system** — `Option` / `Result` : absence and failure are in the signature
- **Type Driven Development** — illegal states are unrepresentable
- **Crates dependency enforcement**
- **Safe concurrency** — compile-time data-race freedom

Also in the toolbox

- **Clippy** — type-aware lints on the compiled program. Unlike syntax-only linters (RuboCop), it knows your types, ownership and data flow
- **Property tests** — `proptest` / `quickcheck` : assert invariants over generated inputs
  - *fees never make a total negative*
  - *rounding is stable across currencies*

Correctness moves from *"we hope we tested it"* → *"it cannot be expressed"*.

Maintainability — productivity & refactoring

Slow prototyping?

Historically yes — the borrow checker slows the first draft.

Not really anymore, thanks to AI assistance and mature crates.

Fast refactoring

Fearless refactoring: change a type, follow the compiler errors, done.

In dynamic languages, a large rename is a test-suite-and-prayer exercise.

The real cost isn't *time to first prototype* — it's **staying safe on the 500th change**, once the code has grown.

Other advantages vs dynamic languages

- **Predictable latency** — no GC pauses; tail latency stays flat
- **Instant startup** — no interpreter boot or JIT warm-up; fast autoscaling and CLI cold starts
- **Lower CPU & memory use** — smaller instances, higher density, fewer machines for the same load
- **Single static binary deployment** — no runtime, no bundle step, tiny images

Why Rust and not another typed language?

Similar typed languages: **Haskell** · **F#** · **OCaml** · **Scala** · **Gleam** · **Elm** · **Roc**

- **Great ecosystem** — tons of high-quality crates (web frameworks, DB ORMs, crypto, decimals, async)
- **Great tooling** — `cargo`, `clippy`, `rustfmt`, `rust-analyzer`: one toolchain
- **Great community** — welcoming, documentation-first culture
- **Mainstream** — real hiring pool, real employability, long-term support
- **Gentler on-ramp from OOP** — familiar syntax (dot notation), imperative style allowed (but you think functionally and give up inheritance)
- **Native speed, no runtime** — compiled ahead of time to machine code; almost every other language on this list pays for a garbage collector or a host VM

Other languages offer a similar type system. Few offer the whole package.

AI coding

- AI agents already know **TyDD**, **DDD** and **Onion/Hexagonal architecture** well — you don't need a long `AGENTS.md` to describe your architecture.
- AI agents **know Rust well** — large, high-quality public corpus
- Rust code is **very explicit**, with clear signatures — agents love this
- The **compiler gives good feedback** to the agent: precise errors, suggested fixes
- **Fearless refactoring** helps agents: they can restructure and verify mechanically

The compiler is a **free, fast, deterministic reviewer** inside the agent loop.

Downsides (honestly)

Concern	Reality
Complex language	But "high-level" Rust is not that complex — you don’t need to seek extreme optimisation
Verbosity	Due to Rust syntax, TyDD and the Onion/Hexagonal architecture. But it’s also explicit and clear; the reader never has to guess
Compile time	Real, but improvable: workspace splitting, `cargo check`, faster linkers, caching

Takeaway

Rust lets you encode **financial rules in types**,

so illegal states never reach production — with predictable latency and fearless refactoring.

Thank you

Questions?